# Script de Preprocesamiento de fMRI

https://gemini.google.com/app/d2eb1095ba0aee85

*User prompt: Dame un script para realizar e preprocesamiento de imágenes de resonancia magnética funcional*

Response: Aquí tienes un script completo y funcional en **Python** utilizando **Nipype**, la librería estándar para crear flujos de trabajo de preprocesamiento de resonancia magnética funcional (fMRI). Este script integra herramientas de **FSL** y **FreeSurfer/ANTs** para cubrir los pasos esenciales del procesamiento.

### Prerrequisitos

Asegúrate de tener instalado Python y los paquetes necesarios:

```
pip install nipype nilearn nibabel
```

*(Nota: Requiere tener instalado FSL en el sistema operativo).*

### Script de Preprocesamiento (`fmri_preprocess.py`)

```
import os
import nipype.interfaces.fsl as fsl
import nipype.pipeline.engine as pe
from nipype.interfaces.utility import IdentityInterface

def create_fmri_prep_pipeline(data_dir, output_dir, subject_id):
    # ------------------------------------------------------------------------
    # 1. Configuración de Nodos Principales
    # ------------------------------------------------------------------------

    # Directorio de trabajo e insumos
    func_file = os.path.join(data_dir, subject_id, "func", "sub-01_task-rest_bold.nii.gz")
    struct_file = os.path.join(data_dir, subject_id, "anat", "sub-01_T1w.nii.gz")

    # Nodo 1: Eliminación de los primeros volúmenes (T1 saturation effects)
    trim = pe.Node(fsl.ExtractROI(t_min=4, t_size=-1), name="trim_volumes")
    trim.inputs.in_file = func_file

    # Nodo 2: Corrección de Tiempo de Corte (Slice Timing Correction)
    slice_timing = pe.Node(fsl.SliceTimer(), name="slice_timing")
    slice_timing.inputs.time_repetition = 2.0  # TR en segundos (ajustar a tu secuencia)
    slice_timing.inputs.interleaved = True     # Modificar según la adquisición

    # Nodo 3: Corrección de Movimiento (MCFLIRT)
    motion_correct = pe.Node(fsl.MCFLIRT(save_plots=True, save_mats=True), name="motion_correction")

    # Nodo 4: Extracción de Cerebro Anatómico (BET)
    bet_struct = pe.Node(fsl.BET(robust=True, frac=0.5), name="struct_brain_extraction")
    bet_struct.inputs.in_file = struct_file

    # Nodo 5: Co-registro Funcional a Anatómico (FLIRT)
    coreg = pe.Node(fsl.FLIRT(dof=6, cost="corratio"), name="coregistration")

    # Nodo 6: Normalización a espacio MNI152
    mni_template = fsl.Info.standard_image("MNI152_T1_2mm_brain.nii.gz")
    normalize = pe.Node(fsl.FLIRT(dof=12), name="normalization")
    normalize.inputs.reference = mni_template

    # Nodo 7: Suavizado Espacial (Spatial Smoothing)
    smoothing = pe.Node(fsl.IsotropicSmooth(), name="spatial_smoothing")
    smoothing.inputs.fwhm = 6.0  # Tamaño del kernel en mm (Full Width at Half Maximum)

    # ------------------------------------------------------------------------
    # 2. Conexión del Flujo de Trabajo (Workflow)
    # ------------------------------------------------------------------------
    wf = pe.Workflow(name="fmri_preprocessing_pipeline", base_dir=output_dir)

    wf.connect([
```

```
        (trim, slice_timing, [("roi_file", "in_file")]),
        (slice_timing, motion_correct, [("slice_time_corrected_file", "in_file")]),
        (bet_struct, coreg, [("out_file", "reference")]),
        (motion_correct, coreg, [("out_file", "in_file")]),
        (coreg, normalize, [("out_file", "in_file")]),
        (normalize, smoothing, [("out_file", "in_file")])
    ])

    return wf

if __name__ == "__main__":
    # Configura tus rutas
    DATA_DIR = "/ruta/a/tu/dataset"
    OUTPUT_DIR = "/ruta/a/tu/salida"
    SUBJECT = "sub-01"

    pipeline = create_fmri_prep_pipeline(DATA_DIR, OUTPUT_DIR, SUBJECT)
    pipeline.run()
```

### Pasos ejecutados por el flujo

- **Recorte inicial:** Remueve los primeros volúmenes (usualmente 4) para asegurar estabilidad de la magnetización.
- **Slice Timing Correction:** Alinea temporalmente las rodajas (*slices*) de cada volumen funcional.
- **Corrección de movimiento:** Utiliza el algoritmo MCFLIRT para realinear los volúmenes a una referencia central y calcula los parámetros de desplazamiento ($x, y, z$, rotaciones).
- **Extracción de cráneo (BET):** Aísla el tejido cerebral de la imagen anatómica T1.
- **Co-registro:** Alinea la imagen funcional del sujeto con su respectiva imagen estructural T1 (6 grados de libertad).
- **Normalización:** Transforma el volumen al espacio estándar MNI152 (12 grados de libertad).
- **Suavizado espacial:** Aplica un filtro gaussiano (FWHM de 6 mm) para incrementar la relación señal-ruido.

---

*User prompt: ¿Me puedes dar un comando para ejecutar el contenedor oficial de fMRIPrep con Docker/Singularity?*

Response: Aquí tienes las dos formas de ejecutar el contenedor oficial de **fMRIPrep**, que es el estándar actual en la comunidad de neuroimagen por garantizar la reproducibilidad.

### Opción 1: Docker (Recomendado con `fmriprep-docker`)

La forma más sencilla y segura de ejecutar fMRIPrep en Docker es usando su wrapper en Python (`fmriprep-docker`), el cual genera y gestiona el comando de Docker automáticamente con las rutas y permisos correctos.

**1. Instalar la CLI wrapper:**

```
pip install fmriprep-docker
```

**2. Comando de ejecución:**

```
fmriprep-docker /ruta/a/dataset_bids /ruta/a/salida \
  participant \
  --participant-label 01 \
  --fs-license-file /ruta/a/license.txt \
  --output-spaces MNI152NLin2009cAsym:res-2 \
  --nthreads 8 --omp-nthreads 4 \
  --mem-mb 16000
```

### Opción 2: Singularity / Apptainer (Ideal para Clusters y HPC)

En entornos de alto rendimiento (HPC) donde no se tienen permisos de superusuario para Docker, se utiliza Singularity.

**1. Descargar la imagen `.sif`:**

```
singularity build fmriprep-23.2.0.simg docker://nipreps/fmriprep:23.2.0
```

**2. Comando de ejecución:**

```
singularity run --cleanenv \
  -B /ruta/a/dataset_bids:/data:ro \
  -B /ruta/a/salida:/out \
  -B /ruta/a/license.txt:/license.txt:ro \
  fmriprep-23.2.0.simg \
  /data /out participant \
  --participant-label 01 \
  --fs-license-file /license.txt \
  --output-spaces MNI152NLin2009cAsym:res-2 \
  --nthreads 8 --omp-nthreads 4 \
  --mem_mb 16000
```

### Parámetros clave a considerar

- **`/dataset_bids`**: La carpeta de entrada **debe** seguir estrictamente la estructura del estándar BIDS.
- **`--fs-license-file`**: **Requisito obligatorio**. Necesitas proporcionar la licencia gratuita de FreeSurfer (`license.txt`), la cual puedes descargar solicitándola en la página web oficial de FreeSurfer.
- **`--output-spaces`**: Define el espacio de normalización de salida (por ejemplo, `MNI152NLin2009cAsym:res-2` para resolución de 2mm).
- **`--cleanenv`** *(solo Singularity)*: Evita que variables de entorno del sistema anfitrión interfieran dentro del contenedor.

---